

Solar Fitness — End-to-End QA Report

Solar rooftop assessment platform · FastAPI backend + Next.js 16 frontend · branch `keerthana` · tested 2026-09-11

- **PASS WITH ISSUES — 8 real bugs found and fixed, 0 regressions, 4 items flagged for your review**

907 / 907

backend tests passing

Clean

frontend build + typecheck

8

bugs fixed

1

high-severity fixed

4

flagged, not auto-fixed

How this was tested

No browser-automation tool was available this session, so testing combined three approaches rather than manual clicking:

- **Live API testing** — ran the actual FastAPI server against the real Postgres/PostGIS + Redis dev instances, and exercised signup, login, checks/sites, IDOR, RBAC, rate limiting, and malformed-input paths with real `curl` requests, reading server logs and DB rows directly.
- **Full automated test suites** — backend `pytest` (907 tests), frontend `next build`, `tsc --noEmit`, `eslint`.
- **Static code review** — every router file, the frontend API client layer, forms, and role-gating logic, cross-checked field-by-field against what the live server actually returned.

Not covered: actual rendered UI (layout, responsiveness, click flows, loading spinners) — flagged below as needing manual/browser verification.

Bugs found and fixed

Hardcoded test login credentials shipped in the JS bundle HIGH FIXED

`app/(auth)/login/page.tsx`, `login/vendor/page.tsx`, `login/admin/page.tsx`, `components/forms/LoginForm.tsx`

STEPS TO REPRODUCE

View page source / JS bundle for `/login`, `/login/admin`, `/login/vendor` in a production build.

EXPECTED

No real credentials present in client-shipped code.

ACTUAL

`admin@test.local / AdminTest123!`, `vendor@test.local / VendorTest123!`, `customer@test.local / CustomerTest123!` were hardcoded as literal props with a one-click "Autofill" button. Confirmed live via `curl` that these accounts exist and work against the dev DB.

ROOT CAUSE

Convenience dev-login shortcut with no build-time gate — the literal strings are bundler-visible regardless of any runtime `NODE_ENV` check inside the component, because they were passed in from the parent page as unconditional string literals.

FIX APPLIED

Two layers: (1) `LoginForm.tsx` only uses/renders the autofill values when `NODE_ENV !== "production"`; (2) more importantly, moved the actual credential strings out of source entirely into `NEXT_PUBLIC_DEV_*` env vars in the gitignored `.env.local` — a build without those vars set (i.e. any real production build) never has the strings in source to begin with.

RETEST

Ran a full production build with `.env.local` removed and grepped the entire `.next` output for all three passwords/emails — zero matches. Rebuilt normally afterward; dev autofill still works.

Vendor could revert a completed job submission by accept → decline MED-HIGH FIXED

`routers/app_vendor.py` — `accept_job` / `decline_job` / `start_job`

STEPS TO REPRODUCE

Vendor completes a job (status `submitted`, with real `measuredCapacityKwp` / `obstacleSurvey` data) → same vendor calls `POST /app/vendor/jobs/{id}/accept` then `/decline`.

EXPECTED

A terminal/submitted job should reject an out-of-order status transition.

ACTUAL

Both calls returned `200` and silently reverted the job to `declined`, discarding the completed submission's terminal state — no auth bypass needed, the assigned vendor's own account could do this to their own job.

ROOT CAUSE

`repositories/vendors.py::update_job_status` / `decline_job` overwrite `row.status` unconditionally; nothing above them guarded the state machine (unlike `app_assessments.py`'s `admin approve/reject`, which already had correct 409 guards).

FIX APPLIED

Added `_ensure_job_status()`, mirroring the existing 409 pattern: `accept` requires `queued`; `decline` / `start` require `queued` / `accepted`.

RETEST

New regression tests `test_accept_already_submitted_job_is_rejected`, `test_decline_job_preserves_it_with_a_declined_status` added and passing; full suite green (907/907).

Unbounded address / out-of-range lat-lng crashed the check-creation endpoint MEDIUM

FIXED

routers/app_checks.py::NewCheckInput, routers/app_sites.py::AppSiteCreate

STEPS TO REPRODUCE

POST /app/checks with a 10,000-character address, or with lat=999, lng=999.

EXPECTED

Clean 422 validation error.

ACTUAL

Long address → unhandled 500 Internal Server Error (plain text, no JSON body — a raw pydantic ValidationError escaping from deep inside the handler). Out-of-range lat/lng → silently accepted as 201, then PostGIS wrapped the invalid coordinates into a nonsensical stored point (lat=-81, lng=-81 for input 999, 999) — silent data corruption, not a crash.

ROOT CAUSE

address had no max_length even though it's reused verbatim as the internal SiteCreate.name, which does have one (255) — the mismatch surfaced as an uncaught exception one layer down instead of a validation error at the API boundary. lat / lng had no range constraint at all, unlike the identical fields elsewhere in the same file which already used Field(ge=-90, le=90).

FIX APPLIED

address: Field(min_length=1, max_length=255), lat: Field(ge=-90, le=90), lng: Field(ge=-180, le=180) on both models.

RETEST

Both cases now return clean 422 with a field-specific message; full suite green.

Customer profile name could be blanked out LOW FIXED

routers/app_checks.py::CustomerProfileUpdate

STEPS TO REPRODUCE

PATCH /app/customer/profile with {"name": ""}.

ACTUAL

Accepted, silently blanking the account's display name.

FIX APPLIED

Field(min_length=1, max_length=255).

Signup form's password minimum didn't match the backend's LOW FIXED

components/forms/SignupForm.tsx vs. routers/app_auth.py::MIN_PASSWORD_LENGTH

STEPS TO REPRODUCE

Enter a 6-7 character password on the signup form.

ACTUAL

Client-side zod schema allowed 6 chars and let the form submit; the backend requires 8 and rejected it — user only finds out after submitting, and (before the next fix below) would have seen a raw JSON error blob

instead of a clear message.

FIX APPLIED

Raised the zod schema's minimum to 8 to match `MIN_PASSWORD_LENGTH`, with the message updated to match.

Backend validation errors displayed as raw JSON instead of readable text

MEDIUM

FIXED

`lib/api/fetchClient.ts::extractErrorMessage` — affects every form on every 422 response app-wide

STEPS TO REPRODUCE

Trigger any backend field-validation error (invalid email, too-short password, out-of-range value, etc.) anywhere in the app.

ACTUAL

FastAPI's 422 body is a JSON array of `{loc, msg, type}` objects — the frontend passed that whole array through `JSON.stringify()` and displayed it verbatim to the user, e.g.

```
[{"type": "string_too_short", "loc": ["body", "password"], "msg": "String should have at least 8 characters", ...}]
```

FIX APPLIED

Added `describeValidationErrors()`, which turns that array into `"password: String should have at least 8 characters"`-style readable text, falling back to the old JSON dump only if the shape doesn't match what's expected.

RETEST

Verified against a live 422 response from the running backend; typecheck/lint clean.

Expired/invalid session left the user stuck on a broken page instead of being sent to login

MEDIUM

FIXED

`lib/api/fetchClient.ts::apiFetch / apiUpload`

STEPS TO REPRODUCE

Let a session token expire (or become invalid) while mid-workflow, then make any API call.

ACTUAL

The client cleared the stored session on a 401 but never navigated anywhere — the user was left looking at a stale/broken page until they happened to navigate somewhere that triggered `AuthGuard`'s next-mount check.

FIX APPLIED

Added `handleUnauthorized()`: clears the session and force-navigates to `/login` on any 401, *except* the login/signup endpoints themselves (where a 401 legitimately means "wrong password," not "session expired," and redirecting away would break the login form's own error display).

RETEST

Typecheck and lint clean (added a scoped eslint-disable with rationale for the intentional hard navigation, since this file has no access to `useRouter`).

Flagged for your review (not changed)

ITEM	WHERE	WHY FLAGGED INSTEAD OF FIXED
ALLOW_HEADER_TENANT config risk	backend/.env	This local <code>.env</code> has it set <code>true</code> . It's an explicitly-documented dev-only escape hatch (default <code>False</code> in <code>config.py</code>) that lets any caller impersonate any tenant on the legacy <code>/sites</code> API via an <code>X-Owner-Orig</code> header with zero authentication — confirmed live with <code>curl</code> . Not exploitable as long as it stays off in any real deployment; flagging because a config value, not code, is what's standing between this and a real vulnerability. <code>.env</code> is gitignored, so this is local-only.
4 pre-existing <code>react-hooks/set-state-in-effect</code> lint errors	AuthGuard.tsx, HeroSection.tsx, UserMenu.tsx, AssessmentReviewClient.tsx	Each looks like a legitimate "hydrate from storage on mount" pattern rather than a real bug, but AuthGuard's redirect logic specifically is worth a second look for a possible flash-of-wrong-content race — can't confirm without a browser. Editing effect timing without being able to visually test the result risks introducing an auth-flow regression, which is exactly the kind of change that should get your sign-off first.
<code>/login/admin</code> and <code>/login/vendor</code> dev-shortcut routes exist in every build	app/(auth)/login/{admin,vendor}/page.tsx	Credentials are no longer embedded (fixed above), but the routes themselves — explicitly titled "(dev)" — still exist and are reachable in a production build. Worth deciding whether to exclude them entirely from prod builds; didn't do this unilaterally since it changes routing/build config, not just a value.
59 ruff lint findings	backend, mostly test files	52 are auto-fixable import-sort/deprecated-typing-syntax noise; the rest (mutable Pydantic defaults, one blind <code>except Exception: continue</code>) were individually reviewed and are safe as-is (Pydantic deep-copies mutable defaults per-instance; the blind <code>except</code> is deliberate "degrade honestly, never crash on a malformed obstacle polygon"

behavior, already documented in that function's own docstring). Not worth a mechanical `--fix` run mixed into a QA pass.

What was verified working correctly

Backend

- Auth: JWT (HS256), 7-day expiry, roles hardcoded server-side on signup (no client-supplied role escalation)
- Login rate limiting: 10/min per email, `429` + `Retry-After` on the 11th attempt
- Anti-enumeration: identical error for unknown email vs. wrong password
- IDOR: cross-tenant access to another user's check/site returns clean `404`, not a leak or a `403`
- Vendor job ownership scoping blocks cross-vendor access
- RBAC: every admin/vendor route uses a real FastAPI `Depends(require_role(...))`; spot-checked ~10 endpoints, all returned clean `403` for the wrong role
- SQL injection: `Robert');`; `DROP TABLE sites;--` stored as a literal string via the ORM; no raw string interpolation found
- Mass assignment: extra fields like `"role": "admin"` in a request body are silently ignored by Pydantic
- CORS: explicit origin allowlist, not a wildcard, even with credentials on
- Malformed JSON / wrong Content-Type / invalid or nonexistent UUIDs all return clean `4xx` JSON, never a stack trace or unhandled 500
- No hardcoded secrets found in backend source; no raw SQL string interpolation

Frontend

- `next build`, `tsc --noEmit` both clean
- No `console.log` / `debugger` left in source
- No `dangerouslySetInnerHTML` on user data — stored XSS payloads (verified via live DB query) render as inert text, not executed
- Forms consistently use `zod` + `react-hook-form`
- Auth token storage approach is a documented, deliberate tradeoff, not an oversight
- Google Maps key is client-exposed via `NEXT_PUBLIC_*`, which is expected/required for that SDK (not a leak by itself) — just confirm in Google Cloud Console that it's HTTP-referrer restricted, which can't be checked from here

Coverage

PAGES/ROUTES ENUMERATED (41 BUILT ROUTES)

Public: `/` `/login` `/login/admin` `/login/vendor` `/signup` `/onboarding` `/check/new` `/check/start`

Customer: `/home` `/profile` `/checks` `/check/[id]/enquiry` `/boundary` `/obstacles` `/processing` `/result`
`/capture/[siteId]/*`

Vendor: `/vendor/dashboard` `/jobs` `/jobs/[id]` `/installations` `/submissions` `/profile` `/service-area`

Admin: `/admin/dashboard` `/assessments` `/vendors` `/installations` `/reports` `/audit-log` `/configuration`
`/platform/*`

APIS TESTED

~90 endpoints inventoried across `/app/auth`, `/app/checks`, `/app/sites`, `/app/admin/*`, `/app/vendor/*`, `/app/customer`, `/app/geocode`, `/app/imports`, `/app/jurisdictions`, plus legacy `/sites`, `/v1/*`. Live-exercised: signup, login, checks CRUD, boundary edit, enquiry, IDOR/RBAC probes on ~15 endpoints, malformed-input edge cases.

ROLES TESTED

customer (full live flow), admin & vendor (RBAC-boundary testing — confirmed correctly blocked as wrong-role customer; did not walk their own dashboards live since that needs a browser)

DEVICES/BROWSERS

Not tested — no browser automation available this session. Tailwind responsive classes are present throughout layout components but actual rendering at mobile/tablet widths was not visually verified.

Still needs manual/browser verification

- Actual responsive layout at mobile/tablet/desktop widths
- Loading, empty, and error state visual rendering across all pages
- AuthGuard redirect timing / possible flash-of-wrong-content on slow networks
- Full click-through of admin and vendor dashboards, filters, modals, pagination
- Whether the Google Maps API key is properly referrer-restricted (config lives in Google Cloud Console, not in this repo)
- Whether any of the seeded test accounts (`@test.local`) exist in an actual production database — only the local dev DB was checked

Generated during an interactive QA session · backend: 907/907 pytest passing, live-tested via curl against a real Postgres+PostGIS/Redis instance · frontend: clean `next build` + `tsc --noEmit` , 8 pre-existing lint errors unrelated to this session's changes.